C Programming & Data Structures

(Answers – 15 Marks each)

Q1. Define Pointer. Write syntax with example. [15]

Definition

A pointer is a variable that stores the memory address of another variable. It "points to" the location where data is kept.

Syntax

```
data_type *pointer_name;      e.g.  int *ptr;
```

Operators used

- & (address-of) : gives address of a variable
- * (dereference) : gives value at that address

Steps

1. Declare a variable 2. Declare a pointer 3. Store address in pointer 4. Access value using *ptr

Program

```
#include <stdio.h>
int main(void) {
  int number = 10;
  int *ptr = &number;
  printf("Value = %d\n", number);
  printf("Address = %p\n", (void *)ptr);
  printf("Value using pointer = %d\n", *ptr);
  return 0;
}
```

Output: Value = 10, Address = (some address), Value using pointer = 10

Features / Uses

- Dynamic memory allocation
- Call by reference (modify caller's variable)
- Efficient array & string handling
- Used to build linked lists, trees, graphs
- NULL pointer = points to nothing (int *p = NULL;)

Q2. Dynamic memory allocation functions. [15]

Definition

Dynamic memory allocation means reserving memory at run time from the heap, using functions of <stdlib.h>. Size can be changed while program runs.

Functions & Syntax

```
ptr = (type *)malloc(size_in_bytes);
ptr = (type *)calloc(n, size_of_each);
ptr = (type *)realloc(ptr, new_size);
free(ptr);
```

1. **malloc()** – allocates one block, memory is uninitialized (garbage).
2. **calloc()** – allocates n blocks, all initialized to zero.
3. **realloc()** – changes size of already allocated block.
4. **free()** – releases memory back to system (avoids memory leak).

Program

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
  int n = 5, i;
  int *arr = malloc(n * sizeof *arr);
  if (arr == NULL) {
    printf("Memory allocation failed\n");
    return 1;
  }
  for (i = 0; i < n; i++) arr[i] = i + 1;
  for (i = 0; i < n; i++) printf("%d ", arr[i]);
  free(arr);
  return 0;
}
```

Output: 1 2 3 4 5

malloc vs calloc

malloc	calloc

1 argument (total size)	2 arguments (count, size)
Garbage values	Initialized to 0
Slightly faster	Slightly slower

Always check for NULL and call free() after use.

Q3. C program to illustrate structure concept. [15]

Definition

A structure is a user-defined data type that groups variables of different data types under one name. Declared using the keyword **struct**.

Syntax

```
struct tag_name {
  data_type member1;
  data_type member2;
};
struct tag_name var;     // variable
var.member1              // access using dot (.)
```

Program

```
#include <stdio.h>
struct Student {
  int roll_no;
  char name[30];
  float marks;
};
int main(void) {
  struct Student s1 = {1, "Aman", 87.5f};
  printf("Roll number: %d\n", s1.roll_no);
  printf("Name: %s\n", s1.name);
  printf("Marks: %.1f\n", s1.marks);
  return 0;
}
```

Output: Roll number: 1, Name: Aman, Marks: 87.5

Points

- Each member gets separate memory; size > sum of members.

- With a pointer use arrow operator: p->marks
- Can be nested, put in arrays, passed to functions.
- Array holds same type; structure holds different types.

Q4. C program for addition of two matrices. [15]

Condition

Both matrices must have the same order (same rows and columns). Result C[i][j] = A[i][j] + B[i][j].

Algorithm

1. Start 2. Read matrices A and B 3. Loop i, j: sum[i][j] = a[i][j] + b[i][j] 4. Print sum 5. Stop

Program

```
#include <stdio.h>
int main(void) {
  int a[2][2], b[2][2], sum[2][2], i, j;
  printf("Enter 4 elements of first matrix:\n");
  for (i = 0; i < 2; i++)
    for (j = 0; j < 2; j++) scanf("%d", &a[i][j]);
  printf("Enter 4 elements of second matrix:\n");
  for (i = 0; i < 2; i++)
    for (j = 0; j < 2; j++) scanf("%d", &b[i][j]);
  for (i = 0; i < 2; i++)
    for (j = 0; j < 2; j++) sum[i][j] = a[i][j] + b[i][j];
  printf("Sum matrix:\n");
  for (i = 0; i < 2; i++) {
    for (j = 0; j < 2; j++) printf("%d ", sum[i][j]);
    printf("\n");
  }
  return 0;
}
```

Sample Input / Output

A = 1 2 / 3 4, B = 5 6 / 7 8 → Sum = 6 8 / 10 12

Time complexity: $O(m \times n)$.

Q5. Linear vs Non-linear data structures. [15]

Basis	Linear	Non-linear
Arrangement	Sequential, one after another	Hierarchical or network
Relationship	Each element has one previous & one next (except ends)	A node can connect to many nodes
Levels	Single level	Multiple levels
Traversal	Single run, one sequence	Multiple paths (BFS, DFS, inorder...)
Memory use	Often contiguous / simple links	Needs more complex linking
Implementation	Easy	Difficult
Examples	Array, linked list, stack, queue	Tree, graph
Uses	Lists, undo, scheduling	File systems, maps, networks

Q6. Define data structure and its classification. [15]

Definition

A data structure is a way of organizing and storing data so that it can be accessed and modified efficiently.

Classification

```
Data Structures
├── Primitive : int, char, float, pointer
└── Non-primitive
    ├── Linear     : Array, Linked list, Stack, Queue
    └── Non-linear : Tree, Graph
```

1. **Primitive:** basic built-in types.
2. **Linear:** Array (fixed, same type), Linked list (nodes with links), Stack (LIFO), Queue (FIFO).
3. **Non-linear:** Tree (parent-child hierarchy), Graph (vertices & edges).

Also: **Static** (fixed size, e.g. array) and **Dynamic** (size changes at run time, e.g. linked list).

Operations

Traversing, inserting, deleting, searching, sorting, merging.

Q7. Define union and write an example. [15]

Definition

A union is a user-defined data type in which all members share the same memory location. Only one member holds a value at a time. Size = size of the largest member.

Syntax

```
union tag_name {
  data_type member1;
  data_type member2;
};
```

Program

```
#include <stdio.h>
union Data {
  int number;
  float decimal;
  char text[20];
};
int main(void) {
  union Data d;
  d.number = 25;
  printf("Number: %d\n", d.number);
  d.decimal = 3.5f;
  printf("Decimal: %.1f\n", d.decimal);
  return 0;
}
```

Output: Number: 25, Decimal: 3.5

Note: writing d.decimal overwrites d.number. Here sizeof(union Data) = 20 bytes.

Structure vs Union

Structure	Union
Keyword struct	Keyword union
Separate memory per member	Shared memory
Size = sum of members (+padding)	Size = largest member

All members usable together	One member at a time

Q8. Address of an array element with example. [15]

Explanation

Array elements are stored in continuous memory. The array name usually converts to a pointer to its first element (base address). Address of an element is found using & or pointer arithmetic.

Formula

1-D : Address of a[i] = Base + i × sizeof(a[0])

2-D (row-major): Address of a[i][j] = Base + (i × cols + j) × size

Index starts from 0. Example: Base = 1000, int = 4 bytes → address of a[2] = 1000 + 2×4 = 1008.

Program

```
#include <stdio.h>
int main(void) {
  int a[4] = {10, 20, 30, 40};
  printf("Base address: %p\n", (void *)a);
  printf("Address of a[2]: %p\n", (void *)&a[2]);
  printf("Using pointer arithmetic: %p\n", (void *)(a + 2));
  printf("Value at a[2]: %d\n", a[2]);
  return 0;
}
```

Output: a[2] address = base + 8 bytes; value = 30. Here a and &a[0] are same; a + 2 points to the third element. Actual addresses depend on the memory layout.